

ADQUISICION DE DATOS Y SEÑALES ANALOGICAS PARA ANALISIS DE VOLTAJE EN FUNCION AL TIEMPO

Ingeniería Mecatrónica

Luis Fernando Lopez Calle¹
lopez.12636909@gmail.com
Yhoel Rojas Condori²
rojas.12542072@gmail.com
Ing. Vidher W. Bustillos D.³

Resumen

En este trabajo se desarrolló un sistema de adquisición de datos utilizando un Arduino Mega y un potenciómetro como entrada analógica. La señal generada por el potenciómetro fue leída mediante el convertidor analógico–digital (ADC) de 10 bits del Arduino, obteniendo valores digitales en un rango de 0 a 1023. Estos datos fueron enviados a una computadora mediante comunicación serial y procesados utilizando Python en Visual Studio Code. Posteriormente, los valores obtenidos se almacenaron en un archivo en formato CSV para facilitar su análisis. A partir de los datos registrados se realizó la conversión de las lecturas digitales a voltaje considerando el voltaje de referencia del sistema. Finalmente, se generó una gráfica de voltaje en función del tiempo, lo que permitió observar cómo varía la señal al modificar la posición del potenciómetro. Los resultados muestran que es posible implementar un sistema simple de adquisición de datos utilizando herramientas accesibles como Arduino y Python para registrar y visualizar señales analógicas.

Palabras claves: Adquisición de datos, Arduino Mega, ADC de 10 bits, Python, potenciómetro, CSV, grafica.

Scientific Acquisition of analog data and signal for voltaje análisis as a function of time

Abstract

This work presents the development of a data acquisition system using an Arduino Mega and a potentiometer as an analog input. The signal generated by the potentiometer was read through the 10-bit analog-to-digital converter (ADC) of the Arduino, producing digital values in the range

¹ Autor de contacto: Estudiante de Ingeniería Mecatrónica, Universidad Mayor de San Andrés (UMSA), La Paz, Bolivia. Cursa la asignatura MEC-237: Laboratorio de Métodos Numéricos III. Correo electrónico: lopez.12636909@gmail.com

² Autor: Estudiante de Ingeniería Mecatrónica, Universidad Mayor de San Andrés (UMSA), La Paz, Bolivia. Cursa la asignatura MEC-237: Laboratorio de Métodos Numéricos III. Correo electrónico: rojas.12542072@gmail.com

³ Director: Docente de métodos numéricos III Facultad De Ingeniería UMSA.

from 0 to 1023. These data were transmitted to a computer through serial communication and processed using Python in Visual Studio Code. The recorded values were stored in a CSV file to facilitate further analysis. The digital readings were then converted into voltage values considering the reference voltage of the system. Finally, a voltage versus time graph was generated, allowing visualization of how the signal changes when the potentiometer position is modified. The results demonstrate that a simple data acquisition system can be implemented using accessible hardware and software tools such as Arduino and Python to record and visualize analog signals.

Keywords: data acquisition, Arduino Mega, 10-bit ADC, Python, potentiometer, CSV, graph.

Introducción o Antecedentes

En la actualidad, los sistemas de adquisición de datos juegan un papel fundamental en diferentes áreas de la ingeniería, ya que permiten registrar y analizar señales provenientes de distintos sensores o dispositivos electrónicos. Estos sistemas facilitan la medición de variables físicas y eléctricas, convirtiendo señales analógicas en datos digitales que pueden ser procesados por computadoras para su análisis y visualización.

Una de las herramientas más utilizadas para este tipo de aplicaciones es la plataforma Arduino, la cual permite realizar lecturas de señales analógicas mediante su convertidor analógico–digital (ADC). En el caso del Arduino Mega, este convertidor posee una resolución de 10 bits, lo que permite representar valores en un rango de 0 a 1023 dependiendo del voltaje de entrada. Este tipo de sistemas resulta útil para realizar experimentos de laboratorio donde se requiere registrar y analizar el comportamiento de una señal eléctrica en función del tiempo.

Por otra parte, el uso de lenguajes de programación como Python permite procesar los datos obtenidos desde el microcontrolador, almacenarlos en archivos para su posterior análisis y generar representaciones gráficas que faciliten la interpretación de los resultados.

En este contexto, el presente trabajo tiene como objetivo implementar un sistema sencillo de adquisición de datos utilizando un potenciómetro como fuente de señal analógica, un Arduino Mega para la lectura de los datos y Python para el almacenamiento y procesamiento de la información. Finalmente, se analiza la variación del voltaje en función del tiempo mediante la generación de gráficas a partir de los datos registrados.

Desarrollo de la Investigación

Solución propuesta

Para el desarrollo del presente trabajo se implementó un sistema sencillo de adquisición de datos que permite registrar y analizar la variación de una señal analógica utilizando herramientas de hardware y software accesibles. La solución propuesta consiste en utilizar un potenciómetro como generador de una señal analógica variable, la cual es leída por un Arduino Mega mediante su convertidor analógico–digital (ADC).

Los valores obtenidos son enviados a una computadora mediante comunicación serial, donde son procesados utilizando Python en el entorno de desarrollo Visual Studio Code. Posteriormente, los

datos adquiridos se almacenan en un archivo en formato CSV para facilitar su análisis y visualización. Finalmente, los datos registrados se utilizan para generar una gráfica de voltaje en función del tiempo, permitiendo observar el comportamiento de la señal producida por el potenciómetro.

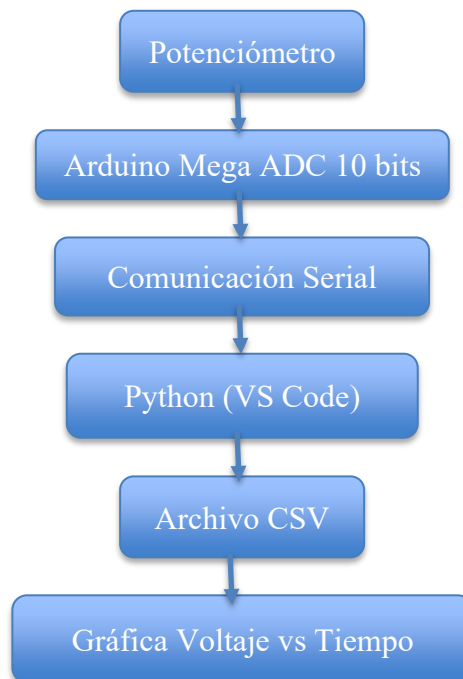
Requerimientos materiales y equipos

Para la implementación del sistema de adquisición de datos se utilizaron los siguientes materiales y herramientas:

- Arduino Mega
- Potenciómetro
- Protoboard
- Cables de conexión
- Computadora personal
- Software Arduino IDE
- Visual Studio Code
- Lenguaje de programación Python
- Librerías de Python para adquisición y procesamiento de datos

Diagrama en bloques

Este diagrama muestra el flujo de información desde la generación de la señal analógica hasta su almacenamiento y representación gráfica.



Codificación de los programas

Para la adquisición y registro de datos se desarrolló un script en Python ejecutado en Visual Studio Code. El programa se encarga de establecer comunicación serial con el Arduino Mega a 9600 baudios, leer de manera continua los valores enviados por el microcontrolador y convertir dichas lecturas del ADC (0–1023) a voltaje utilizando un voltaje de referencia de 5 V.

Adicionalmente, el script almacena los datos en un archivo CSV (tiempo en segundos y voltaje en volts) y genera una gráfica en tiempo real mediante Matplotlib, lo que permite observar la variación del voltaje al modificar la posición del potenciómetro. Para mantener una ventana de visualización estable se utiliza una cola (deque) con un número fijo de muestras, evitando que la gráfica crezca indefinidamente.

Fragmentos principales del código

a) Parámetros de comunicación y configuración general

En esta parte se define el puerto serial, la velocidad de transmisión y el voltaje de referencia utilizado para la conversión de las lecturas del ADC a voltaje.

```
PHYTON

PORT = "COM3"

BAUD = 9600

MAX_POINTS = 200

VREF = 5.0 # Mega/Uno normalmente 5.0V
```

b) Lectura serial y conversión ADC → Voltaje

Luego de leer cada dato por serial, se realiza la conversión a voltaje con la expresión:

$$V = \frac{ADC \cdot V_{Ref}}{1023}$$

```
PHYTON

text = ser.readline().decode("utf-8", errors="ignore").strip()

adc = parse_int_safe(text)

volt = adc * VREF / 1023.0
```

c) Registro de datos en CSV (tiempo y voltaje)

Los datos se guardan en un archivo CSV con encabezado y dos columnas: tiempo (s) y voltaje (V).

```

PHYTON
if CSV_PATH.stat().st_size == 0:
    writer.writerow(["t_seg", "voltaje_V"])

t = time.time() - t0
writer.writerow([round(t, 4), round(volt, 4)])

```

d) Gráfica en tiempo real

Para visualizar la señal de forma dinámica, se implementa una animación con Matplotlib actualizando cada 50 ms.

```

PHYTON
ani = animation.FuncAnimation(fig, update, interval=50, blit=True)
plt.show()

```

Resultados Obtenidos

Durante el desarrollo del experimento se registraron los datos provenientes del potenciómetro utilizando el sistema de adquisición implementado con Arduino Mega y Python. Los valores obtenidos fueron transmitidos mediante comunicación serial hacia la computadora, donde fueron procesados y almacenados en un archivo en formato CSV para su posterior análisis.

El archivo CSV generado contiene dos variables principales: el tiempo de adquisición expresado en segundos y el voltaje correspondiente a cada lectura del potenciómetro. Estos datos permiten analizar el comportamiento de la señal analógica generada por el sensor y su variación a lo largo del tiempo.

Registro de datos experimentales

En la **Tabla 1** se presenta un fragmento representativo de los datos registrados durante el experimento. Estos valores fueron obtenidos directamente del archivo CSV generado por el sistema de adquisición de datos.

Tabla 1. Fragmento de los datos registrados durante la adquisición de la señal del potenciómetro.

t_seg	voltaje_V
70.0314	4.9267
70.0899	4.9413
70.1649	4.9902

70.2268	4.741
70.2905	4.956
70.3578	4.9365
70.4244	4.9267
70.4958	4.9218
70.5592	4.9267
70.6269	4.9267
70.6951	4.9218
70.7796	4.9218
70.845	4.9267
70.9161	4.9267
70.9794	4.9413
71.0443	4.9511
71.1162	4.956
71.1789	4.9609
71.2425	4.9609
71.3142	4.9609
71.3817	4.9658
71.445	4.956
71.516	4.9609
71.5833	4.9658
71.6551	4.9609
71.7206	4.9609
71.7813	4.9609
71.8477	4.9658
71.9151	4.9609
71.9777	4.9609
72.0426	4.9609
72.1097	4.9658
72.1768	4.9707
72.2479	4.9658
72.3104	4.9609
72.3824	4.9658
72.4448	4.824
72.5095	4.6139
72.5794	4.4037
72.6444	4.218
72.709	4.042
72.7789	3.8465
72.8418	3.6804
72.9025	3.4653
72.9658	3.3089
73.0314	3.1476
73.0975	2.9668
73.1612	2.8055

73.2312	2.6491
73.2977	2.4927
73.3571	2.3118
73.4198	2.1261
73.4954	1.9844
73.5571	1.8915
73.6146	1.8915
73.6745	1.8524
73.7491	1.8622
73.8099	1.8964
73.8829	1.8964
73.9456	1.8964
74.0055	1.8964
74.0774	1.7986
74.1487	1.8671
74.212	1.7693
74.2765	1.6325
74.3482	1.4467
74.4139	1.2805
74.4809	1.0753
74.5442	0.8651
74.6087	0.6891
74.6739	0.6061
74.7475	0.6012
74.8129	0.5963
74.8724	0.5963
74.9383	0.5963
74.9986	0.5963
75.0657	0.6012
75.1308	0.6012
75.1904	0.6012
75.2466	0.6012
75.31	0.6012
75.3845	0.6012
75.4476	0.6012
75.5204	0.6012
75.5915	0.6012
75.6617	0.6012
75.7311	0.6012
75.8042	0.6012
75.8671	0.6012
75.947	0.6061
76.0161	0.6012
76.0812	0.6012
76.148	0.6012

76.2224	0.6061
76.2814	0.6012
76.3475	0.6012
76.4046	0.6012
76.4642	0.6061
76.5243	0.6012
76.5963	0.6012
76.6647	0.6061

Nota: Los datos presentados corresponden a las mediciones obtenidas durante el experimento de adquisición de datos utilizando un potenciómetro conectado a un Arduino Mega. Las mediciones fueron registradas mediante un programa desarrollado en Python y almacenadas en un archivo CSV para su posterior análisis.

Representación gráfica de los datos

A partir de los datos almacenados en el archivo CSV se generó una gráfica que representa la variación del voltaje en función del tiempo o del número de muestras registradas. Esta representación gráfica permite visualizar de manera clara el comportamiento de la señal analógica obtenida durante el experimento.

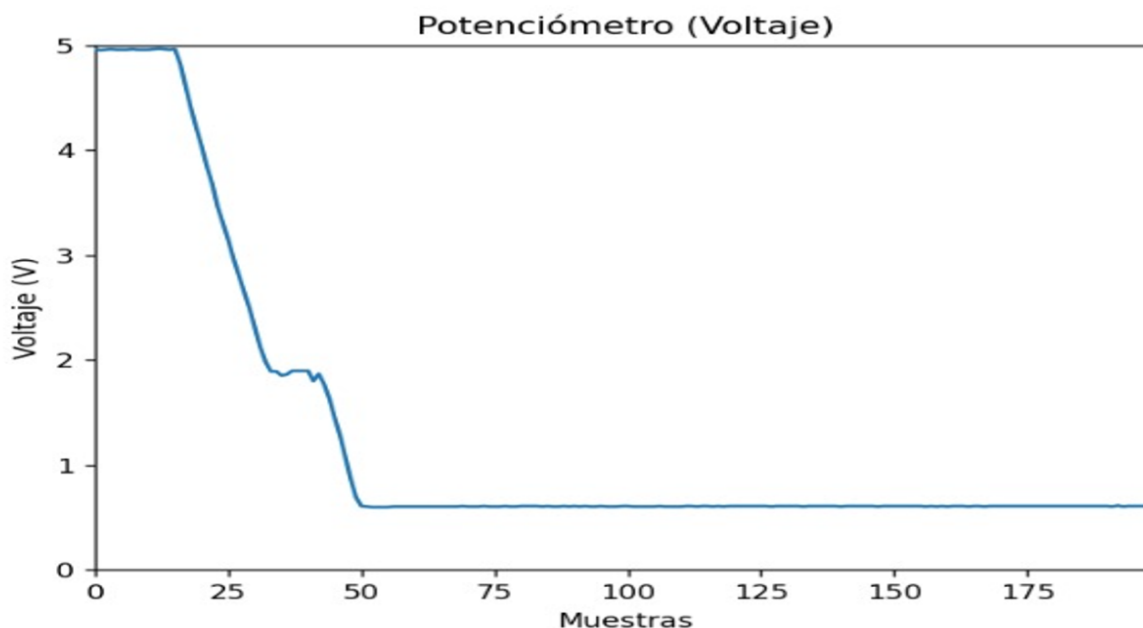


Figura 2. Variación del voltaje del potenciómetro en función de las muestras registradas durante el experimento.

Fuente: Elaboración propia a partir de los datos obtenidos durante el experimento de adquisición de datos utilizando Arduino Mega y Python.

En la gráfica se observa que el voltaje varía dentro de un rango aproximado entre **0 V y 5 V**, dependiendo de la posición del potenciómetro. Durante las pruebas realizadas se registraron cambios progresivos en el voltaje, evidenciando el correcto funcionamiento del sistema de adquisición de datos y la capacidad del programa desarrollado para registrar y almacenar las mediciones en tiempo real.

Los resultados obtenidos muestran que el sistema implementado permite registrar correctamente las variaciones de la señal analógica generada por el potenciómetro, facilitando su posterior análisis mediante herramientas de procesamiento de datos.

Discusión y conclusiones

Discusión de resultados

Los resultados obtenidos durante el experimento muestran que el sistema de adquisición de datos implementado fue capaz de registrar correctamente la variación del voltaje generado por el potenciómetro. A partir de los datos almacenados en el archivo CSV se pudo observar cómo el voltaje cambia de acuerdo con la posición del potenciómetro, lo cual confirma el correcto funcionamiento del circuito y del programa desarrollado para la adquisición de datos.

El comportamiento observado en los resultados es coherente con el funcionamiento esperado de un potenciómetro, ya que este dispositivo actúa como un divisor de voltaje que permite variar la tensión de salida dentro de un rango determinado. En este caso, los valores registrados se encuentran dentro del rango aproximado entre 0 V y 5 V, correspondiente al voltaje de referencia utilizado por el Arduino Mega.

Asimismo, el uso del convertidor analógico–digital (ADC) de 10 bits del Arduino permite obtener valores digitales en un rango de 0 a 1023, los cuales posteriormente fueron convertidos a valores de voltaje mediante una relación proporcional. Este proceso permitió representar la señal analógica en forma digital y analizar su comportamiento mediante herramientas de programación como Python.

El método utilizado demostró ser adecuado para realizar adquisiciones de datos simples en aplicaciones de laboratorio. El uso de Python permitió almacenar los datos en formato CSV y generar representaciones gráficas que facilitan la interpretación de los resultados obtenidos.

Sin embargo, es importante considerar algunas limitaciones del sistema implementado. La resolución del ADC depende del número de bits del convertidor, por lo que las mediciones obtenidas presentan una precisión limitada por la resolución del sistema. Además, factores como ruido eléctrico, variaciones en el voltaje de referencia o la estabilidad del potenciómetro pueden influir ligeramente en las mediciones registradas.

A pesar de estas limitaciones, el sistema implementado cumple adecuadamente con el objetivo del experimento, permitiendo observar y analizar el comportamiento de una señal analógica mediante un sistema básico de adquisición de datos.

Conclusiones

A partir de los resultados obtenidos durante el experimento se puede concluir que el sistema de adquisición de datos desarrollado utilizando Arduino Mega y Python permite registrar de manera adecuada las variaciones de voltaje generadas por un potenciómetro.

El uso del convertidor analógico–digital del Arduino permitió transformar la señal analógica en valores digitales que posteriormente fueron procesados y almacenados en un archivo CSV. Esto facilitó el análisis de los datos obtenidos y la generación de gráficas que representan el comportamiento del voltaje en función del tiempo o de las muestras registradas.

Asimismo, el empleo del lenguaje de programación Python permitió automatizar el proceso de adquisición y almacenamiento de datos, demostrando que es una herramienta útil para el procesamiento y visualización de información experimental.

Finalmente, el experimento permitió comprender el funcionamiento básico de un sistema de adquisición de datos, así como la importancia de convertir señales analógicas en información digital para su posterior análisis en aplicaciones de ingeniería.

Acerca de las citas

En la elaboración de trabajos académicos es fundamental reconocer las ideas y aportes de otros autores que han contribuido al desarrollo del tema investigado. Por esta razón, el uso adecuado de citas permite dar crédito a las fuentes consultadas y evita el uso indebido de información que no es propia. Las citas se utilizan cuando se incorporan conceptos, definiciones o resultados provenientes de libros, artículos científicos u otras fuentes confiables que sirven como base teórica para el desarrollo del trabajo.

Las citas pueden presentarse de manera textual o mediante paráfrasis. En el caso de las citas textuales cortas, estas se incluyen dentro del párrafo entre comillas y acompañadas de la referencia correspondiente del autor y el año de publicación. Cuando la cita supera las cuarenta palabras, se presenta en un párrafo independiente con sangría, permitiendo diferenciar claramente el contenido citado del resto del texto.

Asimismo, todas las fuentes citadas dentro del documento deben aparecer posteriormente en la sección de referencias bibliográficas. Esto permite que el lector pueda identificar y consultar las fuentes utilizadas durante la investigación. El uso correcto de citas y referencias contribuye a fortalecer la credibilidad del trabajo académico y demuestra el sustento teórico que respalda el análisis realizado.

Referencias bibliográficas

Las referencias bibliográficas presentadas en este trabajo corresponden a las fuentes consultadas para comprender los fundamentos teóricos relacionados con la adquisición de datos, el funcionamiento del microcontrolador Arduino y el uso del lenguaje de programación Python para el registro y procesamiento de información. Estas fuentes proporcionan información técnica sobre el convertidor analógico–digital (ADC), la lectura de señales analógicas y las herramientas utilizadas para almacenar y visualizar los datos obtenidos durante el experimento.

Referencias

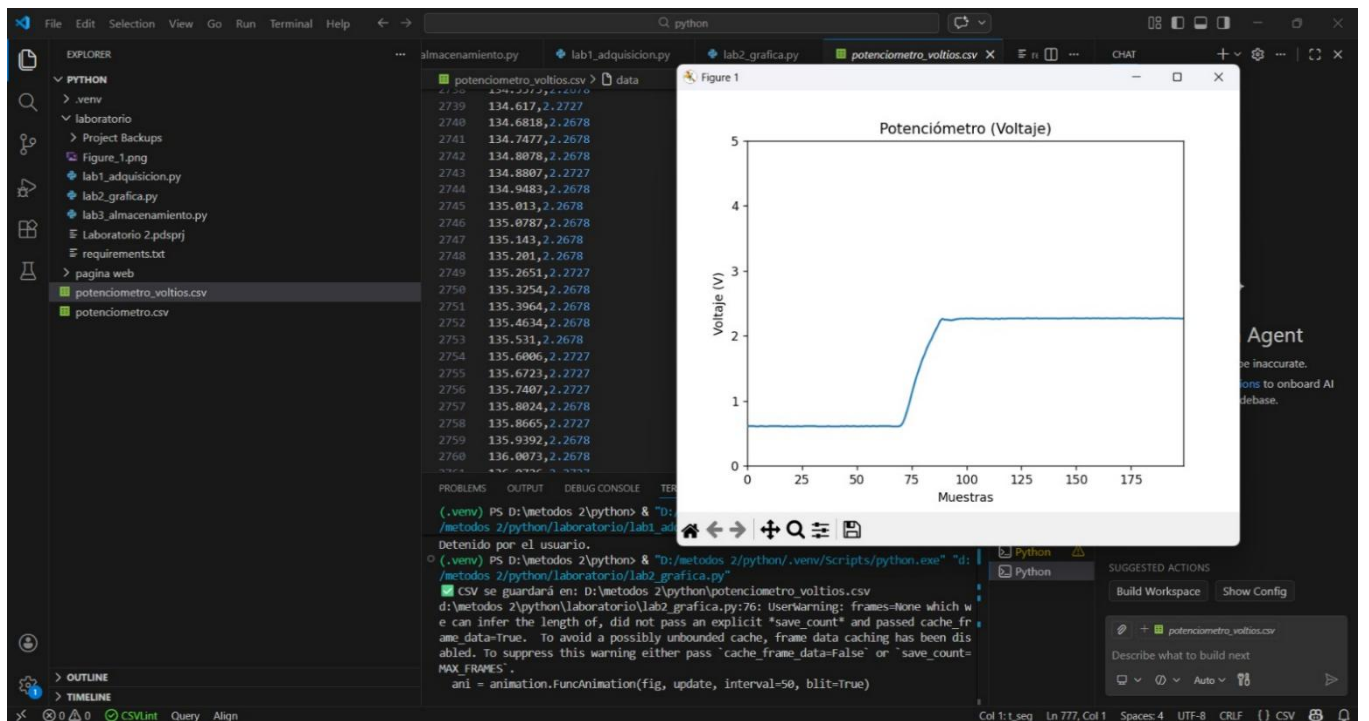
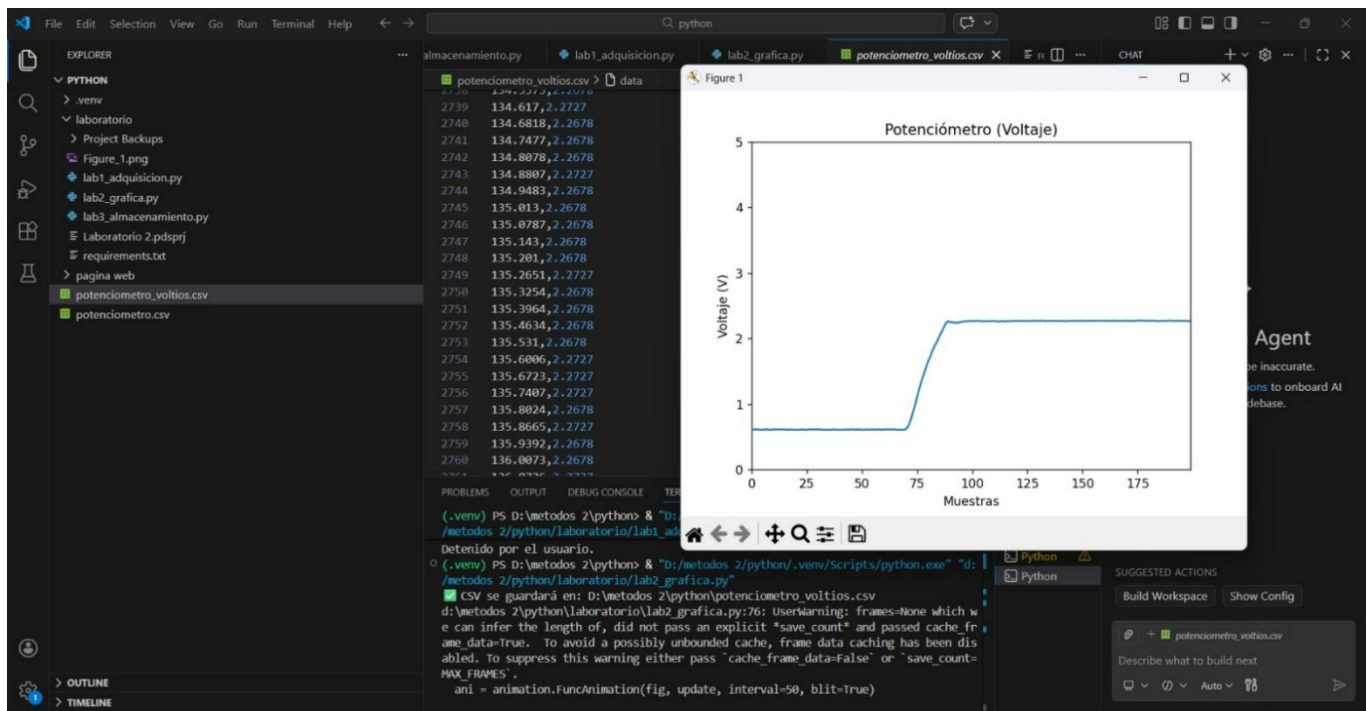
Arduino. (2023). *Arduino Mega 2560 Rev3 – Technical specifications*. Arduino Documentation. <https://www.arduino.cc/en/Main/ArduinoBoardMega2560>

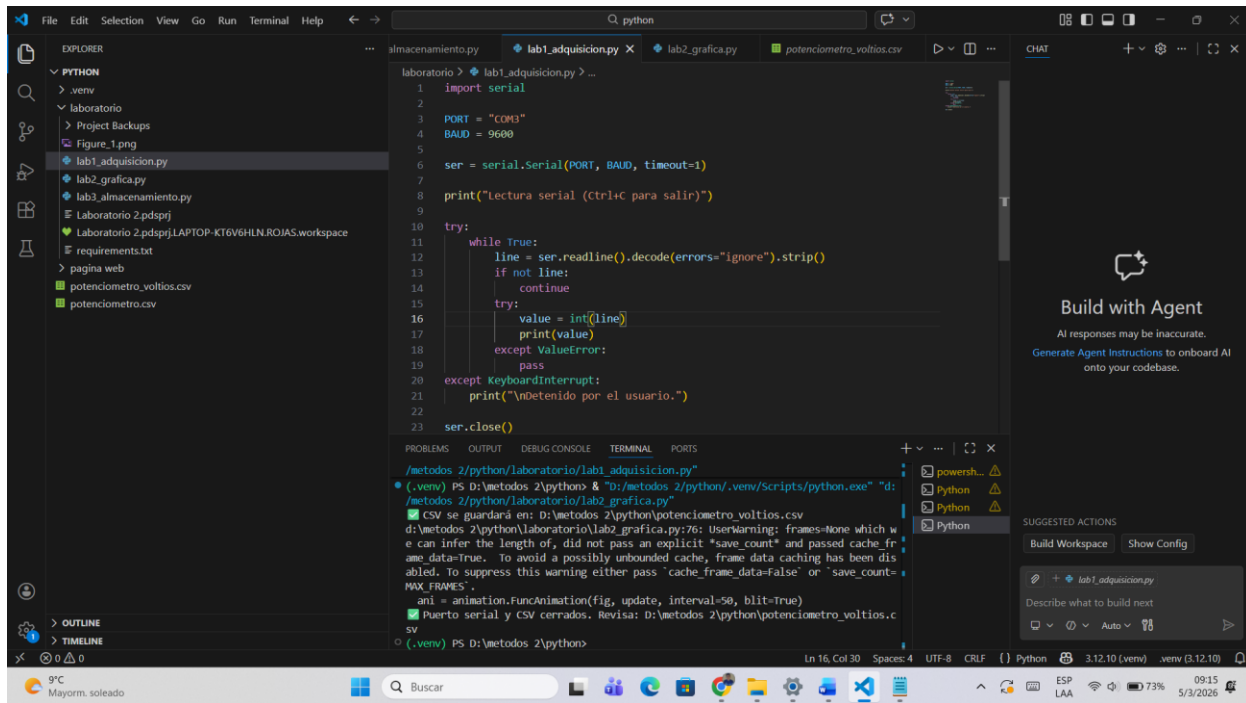
McKinney, W. (2018). *Python for Data Analysis: Data wrangling with Pandas, NumPy, and IPython* (2nd ed.). O'Reilly Media.

Matplotlib Development Team. (2023). *Matplotlib documentation*. <https://matplotlib.org/stable/index.html>

Python Software Foundation. (2023). *Python language reference*. <https://docs.python.org/3/>.

ANEXOS





```
1 import serial
2
3 PORT = "COM3"
4 BAUD = 9600
5
6 ser = serial.Serial(PORT, BAUD, timeout=1)
7
8 print("lectura serial (ctrl+c para salir)")
9
10 try:
11     while True:
12         line = ser.readline().decode(errors="ignore").strip()
13         if not line:
14             continue
15         try:
16             value = int(line)
17             print(value)
18         except ValueError:
19             pass
20 except KeyboardInterrupt:
21     print("\ndetenido por el usuario.")
22
23 ser.close()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

/.venv PS D:\metodos 2\python> & "D:\metodos 2\python\Scripts\python.exe" "d:\metodos 2\python\laboratorio\lab1_adquisicion.py"

CSV se guardará en: D:\metodos 2\python\potenciometro_voltios.csv

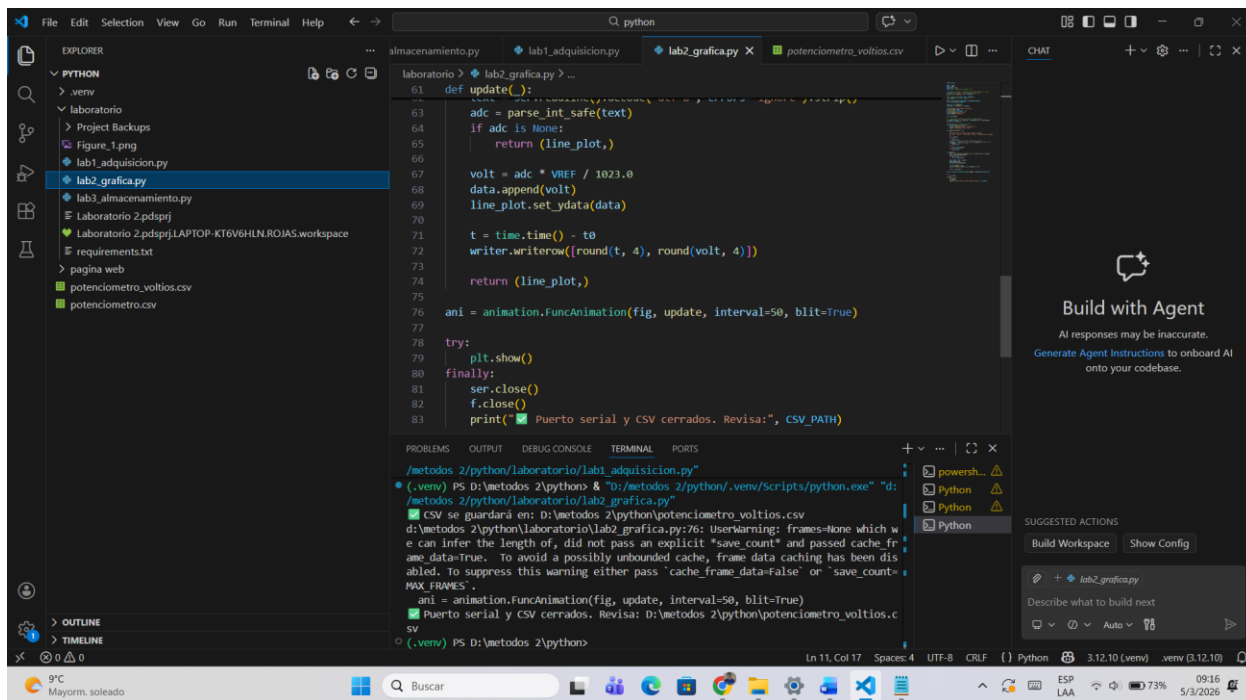
d:\metodos 2\python\laboratorio\lab2_grafica.py:76: UserWarning: frames=None which we can infer the length of, did not pass an explicit "save_count" and passed cache_frame_data=True. To avoid a possibly unbounded cache, frame data caching has been disabled. To suppress this warning either pass "cache_frame_data=False" or "save_count=MAX_FRAMES".

ani = animation.FuncAnimation(fig, update, interval=50, blit=True)

Puerto serial y CSV cerrados. Revisa: D:\metodos 2\python\potenciometro_voltios.csv

(.venv) PS D:\metodos 2\python>

Ln 16, Col 30 Spaces: 4 UTF-8 CRLF Python 3.12.10 (.venv) .venv (3.12.10)



```
61 def update():
62     adc = parse_int_safe(text)
63     if adc is None:
64         return (line_plot,)
65
66     volt = adc * VREF / 1023.0
67     data.append(volt)
68     line_plot.set_ydata(data)
69
70     t = time.time() - t0
71     writer.writerow([round(t, 4), round(volt, 4)])
72
73     return (line_plot,)
74
75 ani = animation.FuncAnimation(fig, update, interval=50, blit=True)
76
77 try:
78     plt.show()
79 finally:
80     ser.close()
81     f.close()
82     print("✓ Puerto serial y CSV cerrados. Revisa:", CSV_PATH)
83
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

/.venv PS D:\metodos 2\python> & "D:\metodos 2\python\Scripts\python.exe" "d:\metodos 2\python\laboratorio\lab2_grafica.py"

CSV se guardará en: D:\metodos 2\python\potenciometro_voltios.csv

d:\metodos 2\python\laboratorio\lab2_grafica.py:76: UserWarning: frames=None which we can infer the length of, did not pass an explicit "save_count" and passed cache_frame_data=True. To avoid a possibly unbounded cache, frame data caching has been disabled. To suppress this warning either pass "cache_frame_data=False" or "save_count=MAX_FRAMES".

ani = animation.FuncAnimation(fig, update, interval=50, blit=True)

Puerto serial y CSV cerrados. Revisa: D:\metodos 2\python\potenciometro_voltios.csv

(.venv) PS D:\metodos 2\python>

Ln 11, Col 17 Spaces: 4 UTF-8 CRLF Python 3.12.10 (.venv) .venv (3.12.10)

